

## P6: Working with Multiples

Name: Alston Liow Guan You

Student ID: 37440985

Date: 16 August 2026

### Overview

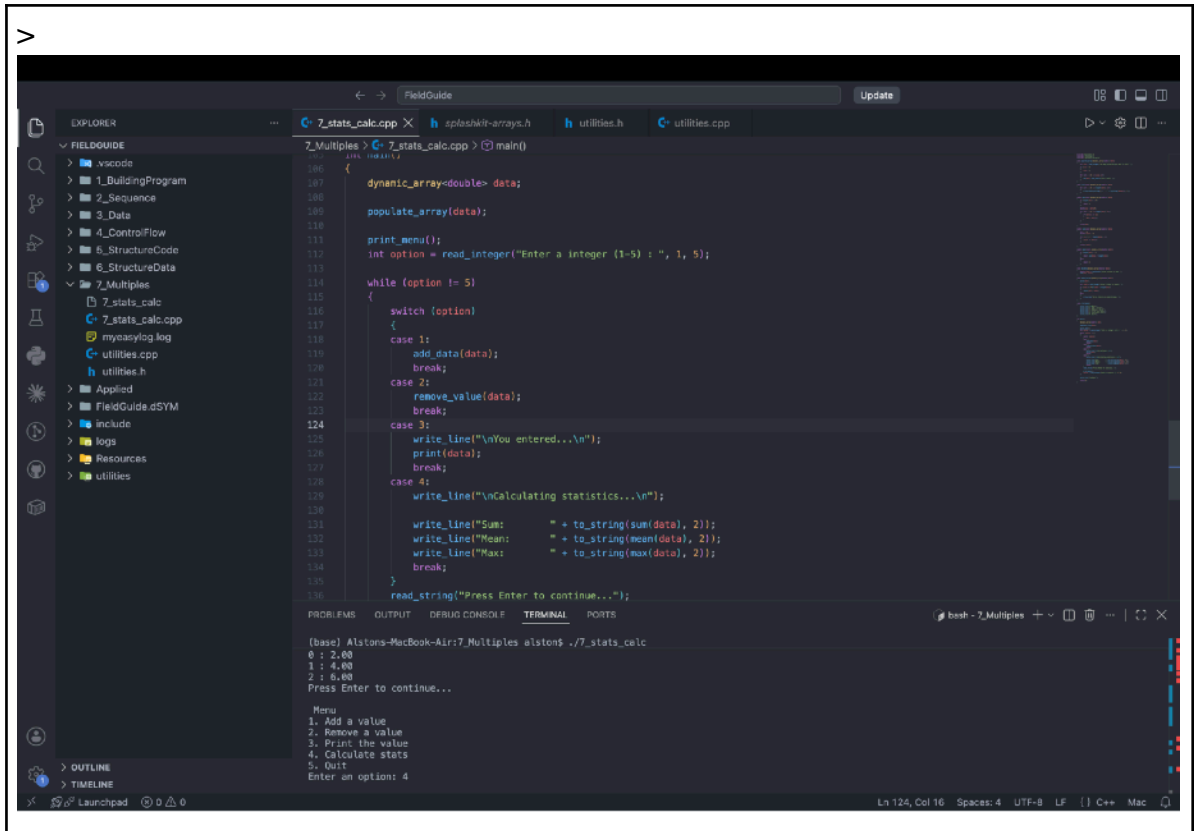
This task is an opportunity to explore and use language mechanisms to allow you to easily work with multiple values, which can grow and shrink in number as the program executes.

#### Tip: Systems thinking

Work to develop your ability to see how this comes together as a system. The parts are not independent of each other. Understanding each part in isolation is a good start. Make sure to follow this up by seriously focusing on seeing how the parts come together and work as a system. When you can see this, you can start to think about how to build and manipulate these systems, getting them to achieve what you want.

### Do the following:

- Everything you need is in the Handling Multiples chapter of our [Programmer's Field Guide](#). The [Intro Tours](#) walk through the concepts, and should give you a good start.
- Add a screenshot image showing you were able to create and work with arrays of simple values - using a dynamic array, using interaction, for performing actions like searching, expanding, removing, and accumulating values. You can use the [statistics calculator tour](#) to achieve this (multiple pages).



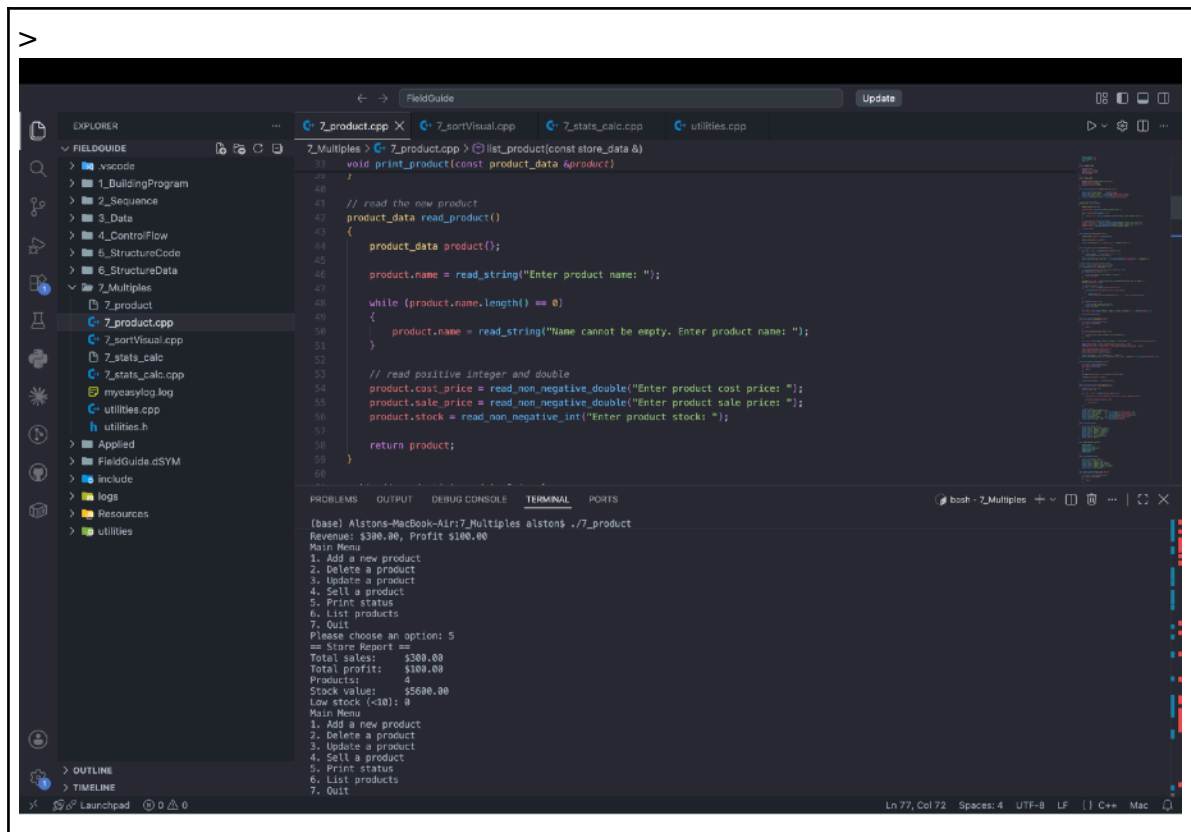
```
>
FieldGuide
7_stats_calc.cpp
106 {
107     dynamic_array<double> data;
108     populate_array(data);
109
110     print_menu();
111     int option = read_integer("Enter a integer (1-5) : ", 1, 5);
112
113     while (option != 5)
114     {
115         switch (option)
116         {
117             case 1:
118                 add_data(data);
119                 break;
120             case 2:
121                 remove_value(data);
122                 break;
123             case 3:
124                 write_line("\nYou entered...\n");
125                 print(data);
126                 break;
127             case 4:
128                 write_line("\nCalculating statistics...\n");
129
130                 write_line("Sum:      " + to_string(sum(data), 2));
131                 write_line("Mean:    " + to_string(mean(data), 2));
132                 write_line("Max:     " + to_string(max(data), 2));
133                 break;
134         }
135         read_string("Press Enter to continue...");
136     }
137 }
```

Terminal Output:

```
(base) Alstons-MacBook-Air:7_Multiples alston$ ./7_stats_calc
0 : 2.00
1 : 4.00
2 : 6.00
Press Enter to continue...

Menu
1. Add a value
2. Remove a value
3. Print the value
4. Calculate stats
5. Quit
Enter an option: 4
```

- Add a screenshot image showing you were able to create and use arrays of your own custom types (structs). You can use the [product database tour](#) to achieve this (multiple pages).

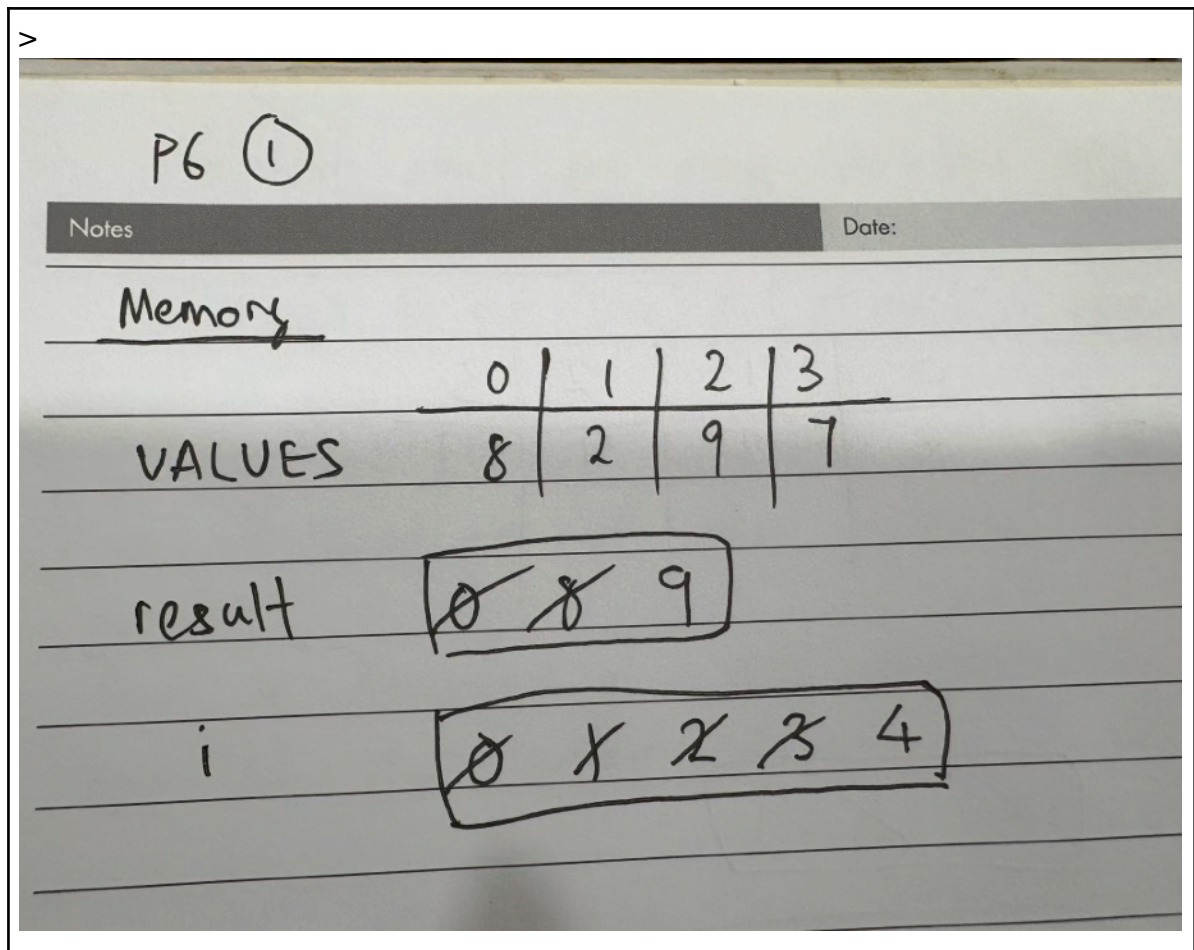


- Add a photo showing you can use our [hand execution process](#) to illustrate how code operates on arrays using the following source code.

```
const double VALUES[4] = { 8, 2, 9, 7 };
const int SIZE = 4;

double result = 0;

// Check the rest of the values
for (int i = 0; i < SIZE; i++)
{
    if (VALUES[i] > result)
    {
        result = VALUES[i];
    }
}
```



- Consider the code as a system - what is it aiming to achieve? It does aim to achieve something meaningful - what is that? Can you see any issues with the way this is working in relation to what it is likely aiming to achieve?

> It is aiming to achieve to find the largest number in the arrays which is positive number.

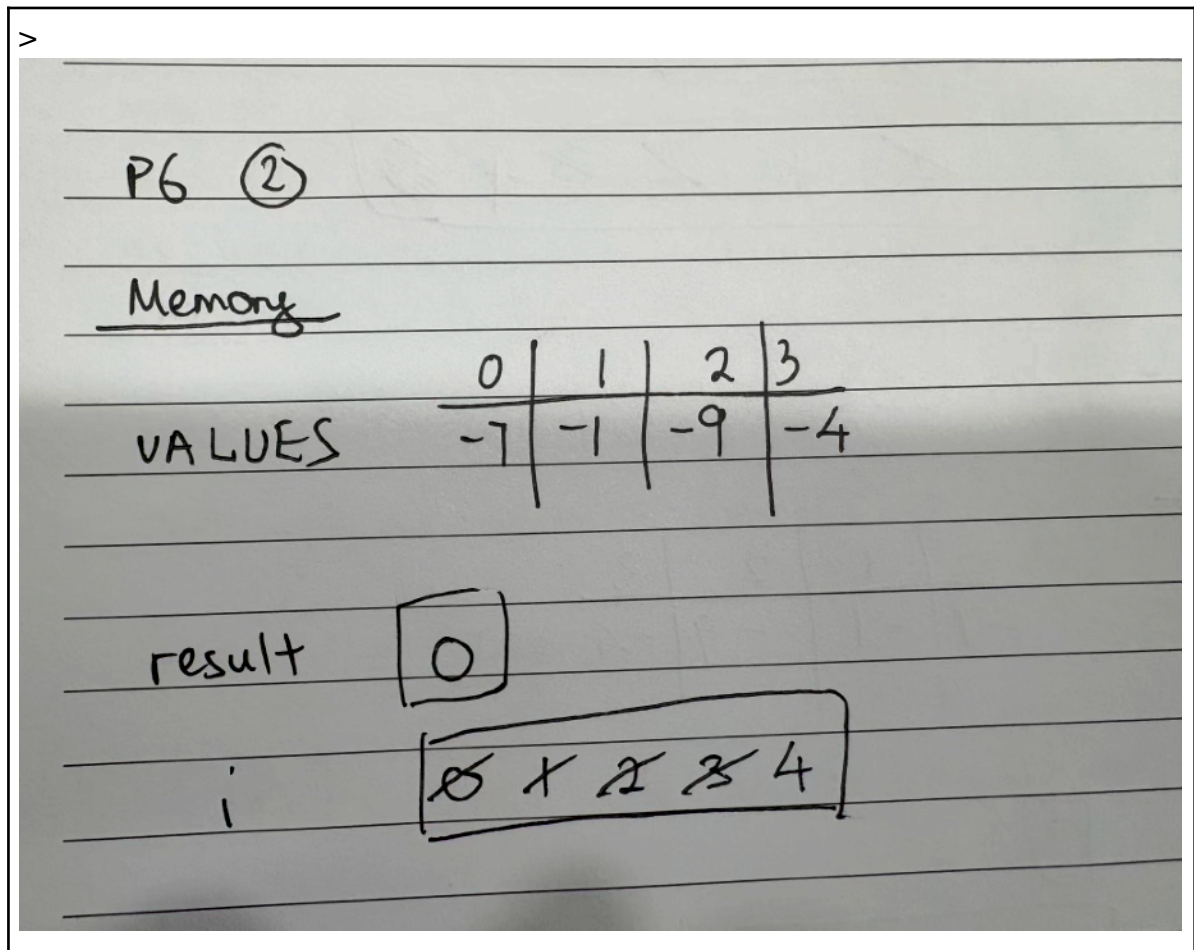
- Repeat the hand execution for the following source code.

```
const double VALUES[4] = { -7, -1, -9, -4 };
const int SIZE = 4;
```

```
double result = 0;
```

```
// Check the rest of the values
for (int i = 0; i < SIZE; i++)
{
    if (VALUES[i] > result)
    {
        result = VALUES[i];
    }
}
```

```
}  
}
```

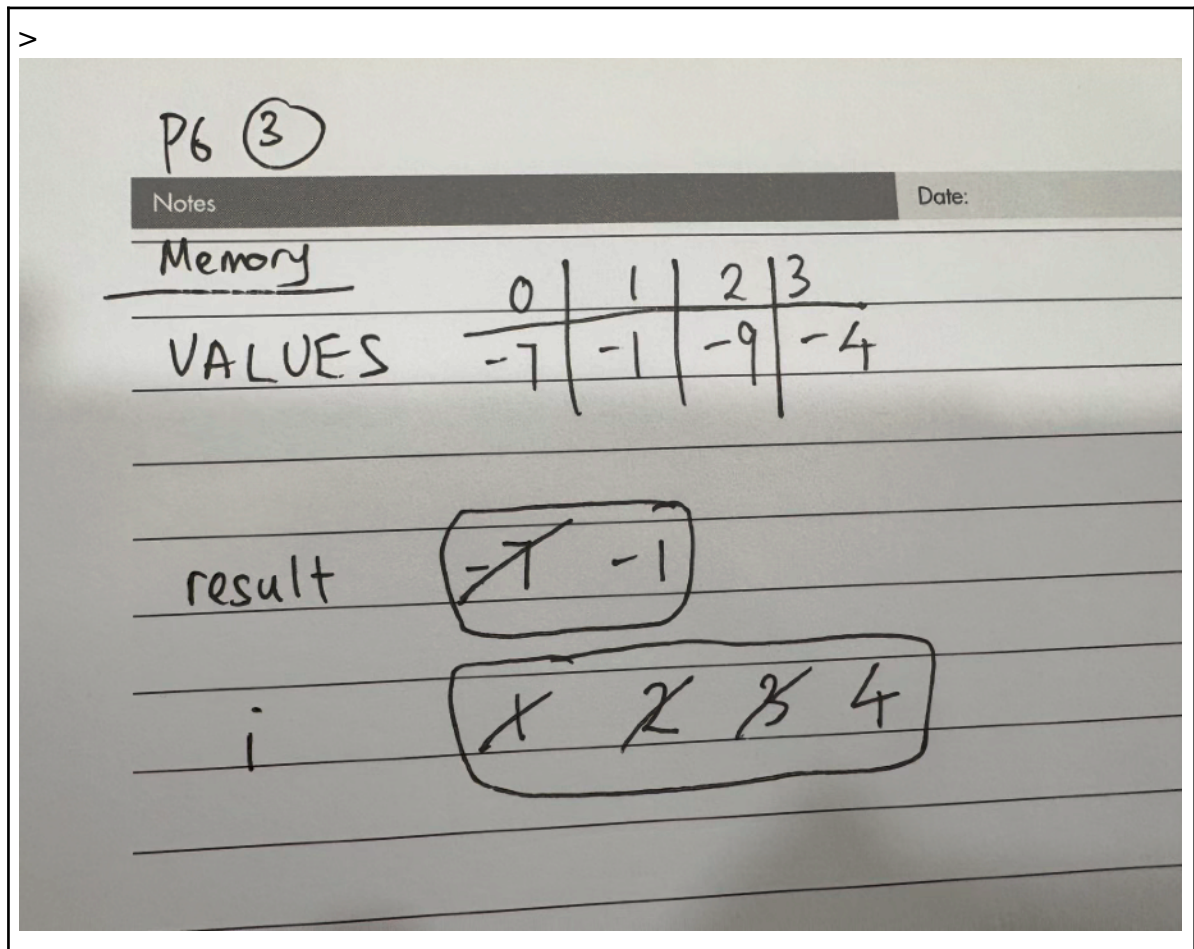


- Repeat the hand execution for the following source code - which made two small changes to the logic!

```
const double VALUES[4] = { -7, -1, -9, -4 };  
const int SIZE = 4;
```

```
double result = VALUES[0];
```

```
// Check the rest of the values  
for (int i = 1; i < SIZE; i++)  
{  
    if (VALUES[i] > result)  
    {  
        result = VALUES[i];  
    }  
}
```



- How did the change impact the result? What does this say about the way these systems are put together? Consider how this impacts cases where others (including AI) are writing the code for you.

> Before changing, the result in the VALUE compare its with 0, after changing, it compare with VALUE[0], which inside the VALUE, so the changing make it can find the largest number inside the Arrays.

### Tip: Focus and attention to details

Computing capabilities require you to be able to be meticulous, pay attention to details, and be able to focus on things for a period of time. These hand execution tasks will exercise these attributes. Make sure you can do this in a distraction free environment, and that you force yourself to focus on just this activity. It is a little mindless, you are wanting to show you *know* what the computer does. But you need to be able to step through things at this level to really see what the computer is doing - it is unintelligent, so we are relying on your intelligence to make it do the right thing!

- Now you need to demonstrate you can perform hand execution on more complex code. You should be able to hand-execute code without understanding the algorithm itself - the computer is unintelligent and it can follow the steps.

```
const int MAX_SIZE = 5;

double data[MAX_SIZE] = {5, 3, 2};
int size = 3;

double temp;
for (int i = 0; i < MAX_SIZE - 1; i++)
{
    for (int j = 0; j < size - i - 1; j++)
    {
        if (data[j] > data[j + 1])
        {
            temp = data[j];
            data[j] = data[j + 1];
            data[j + 1] = temp;
        }
    }
}
```

P6 (4)

Notes \_\_\_\_\_ Date: \_\_\_\_\_

VALUES = 

|              |              |              |              |
|--------------|--------------|--------------|--------------|
| <del>8</del> | <del>2</del> | <del>9</del> | <del>7</del> |
|--------------|--------------|--------------|--------------|

data = 

|              |              |              |   |   |
|--------------|--------------|--------------|---|---|
| 0            | 1            | 2            | 3 | 4 |
| <del>5</del> | <del>2</del> | <del>2</del> |   |   |
| <del>3</del> | <del>5</del> | 5            |   |   |
| 2            | <del>2</del> |              |   |   |
|              | 3            |              |   |   |

temp = 

|              |              |   |
|--------------|--------------|---|
| <del>5</del> | <del>5</del> | 3 |
|--------------|--------------|---|

i 

|              |    |
|--------------|----|
| <del>0</del> | 12 |
|--------------|----|

j 

|              |   |
|--------------|---|
| <del>0</del> | 2 |
|--------------|---|

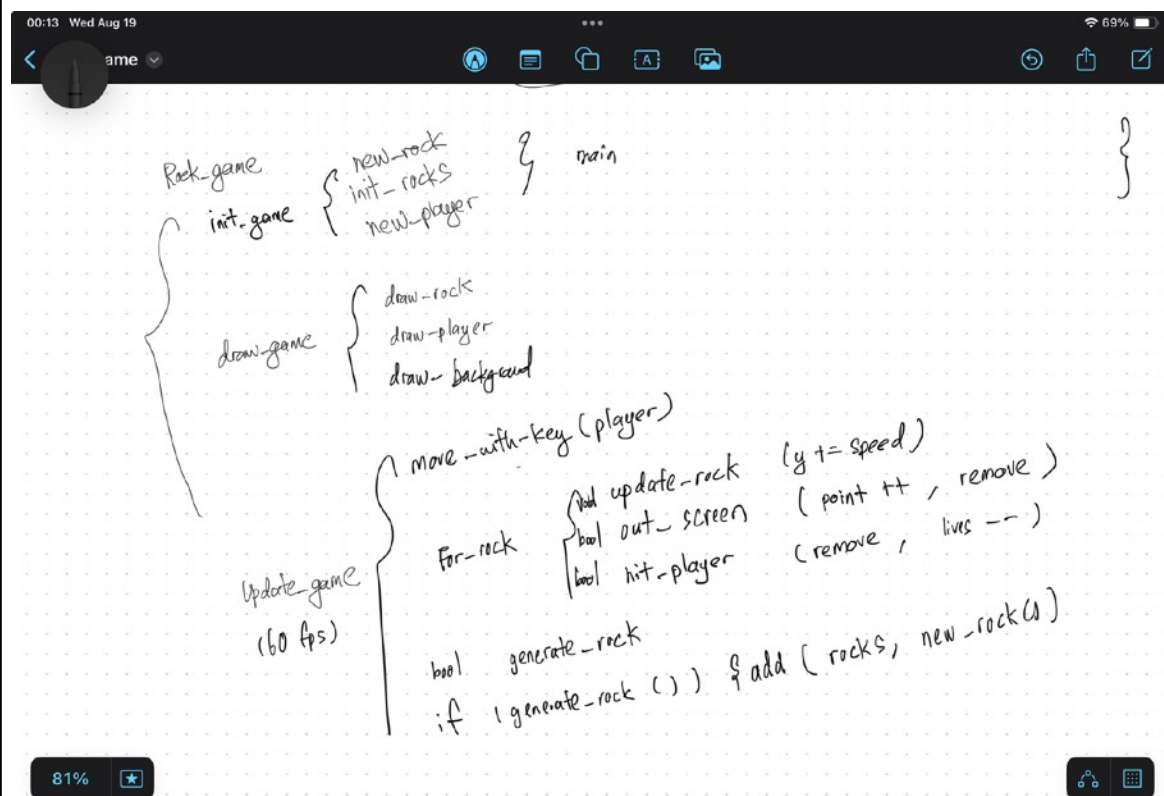
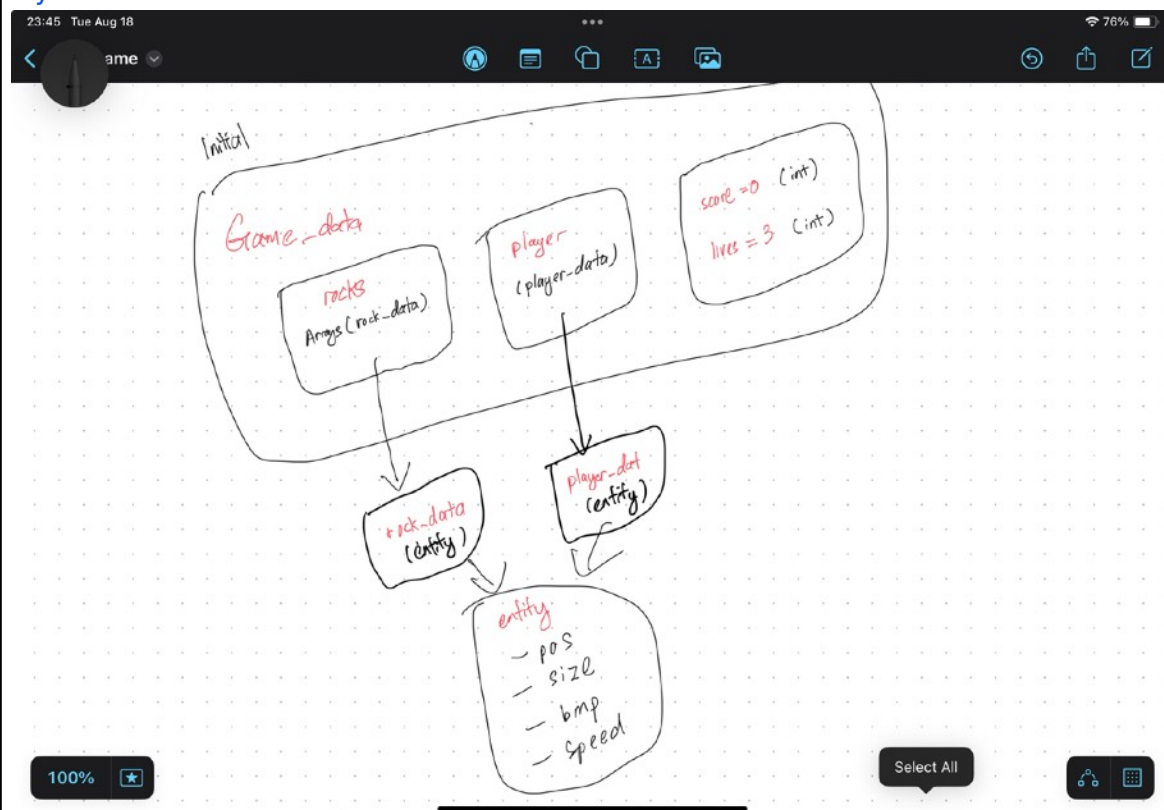
j 

|   |
|---|
| 0 |
|---|

>

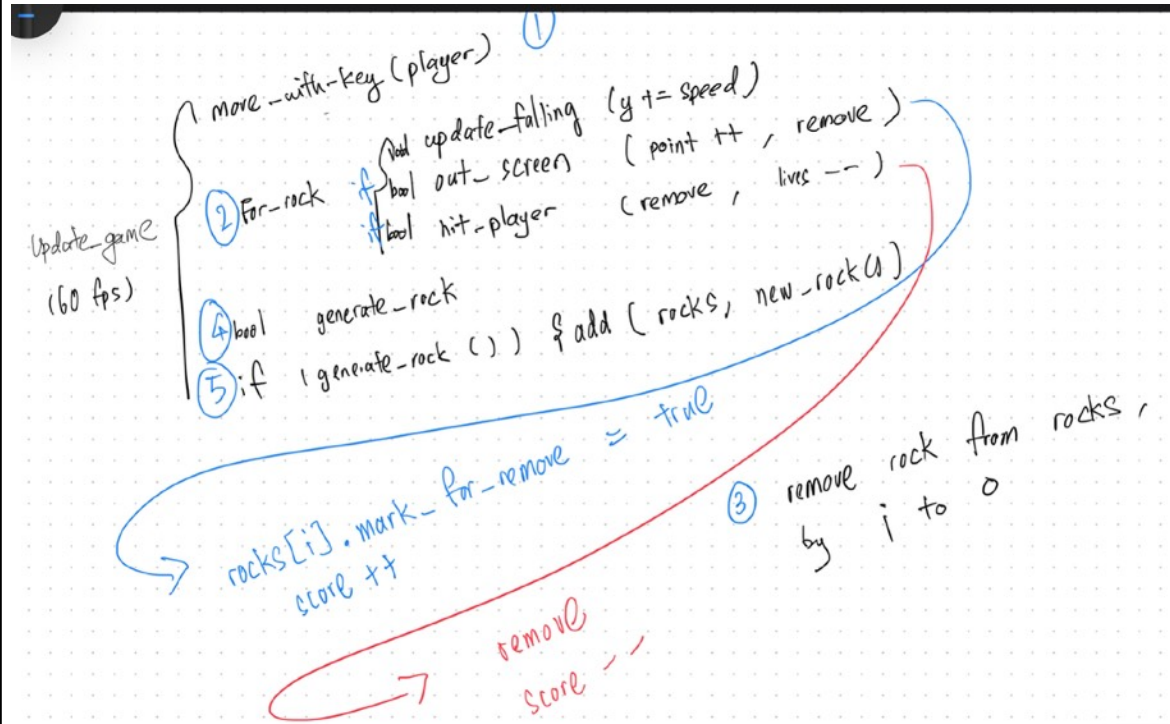
- Complete one of the [Test Your Capabilities](#) tasks from the chapter to see that you can now put these tools together to achieve something yourself.

> I choose the rock game. First I get some reference at the fly catch and map explorer, and plan about the structure of data and function I need to use about this game, this is my initial structure.

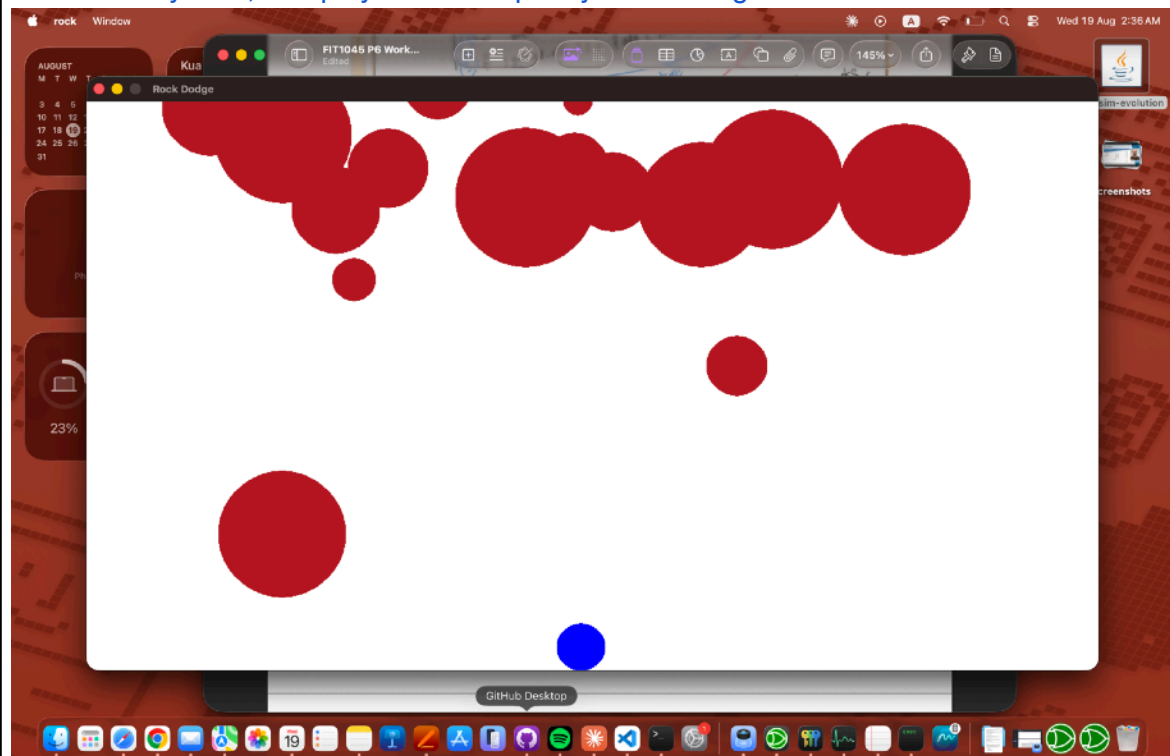


I write all the function step by step until need to combine all update function in to `update_game()`, I get stuck here. I have some idea to add something in this game, for example is typed\_down(SPACE\_KEY) to jump , and I want to add a horizontal thing like the rock. But I just put beside and do the main thing first.

I analyse the `update_game()` like this



First time I complete this game I forget to add a `const int MAX_ROCK_COUNT = 100`, so it is too many rock, and player will die quickly. Then the game become like this



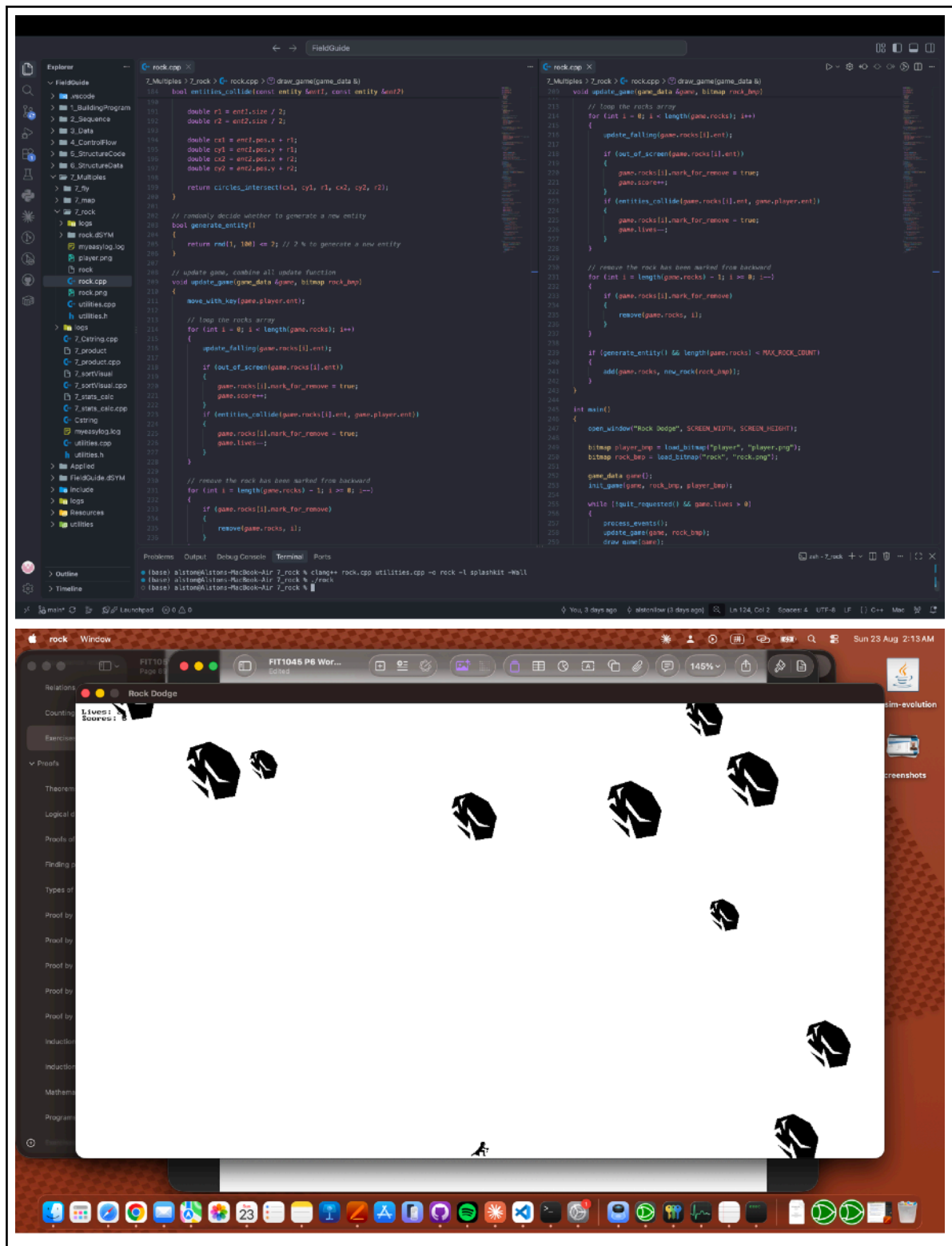
Now I want to change the circle to using sprite to present the animation. I check the splash kit and try to change my code.

When adding a bitmap, is very hassle to change all the function parameter, so I decide to add bitmap into game\_data struct.

The position all get wrong after I change to bitmap and sprite, its very difficult to find the bug, I spend many time and going to have a breakdown.

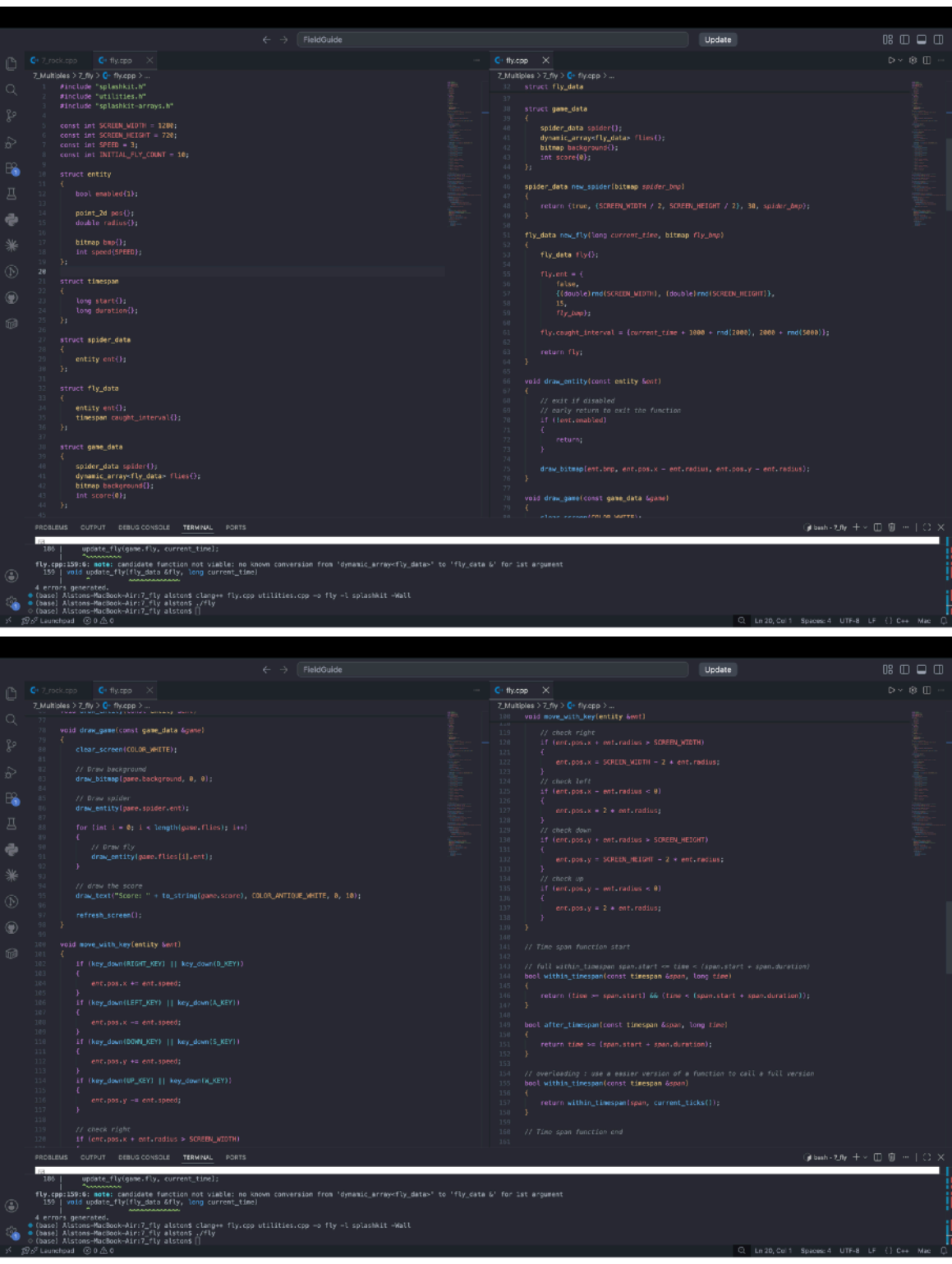
Finally, I fixed it with some calculation.





- If you want to extend your capabilities further in terms of creating larger programs, add a screenshot here showing what you were able to achieve. We recommend both the [extended fly catch tour](#), and the [map explorer tour](#). Fly catch is a quick addition to the program with structs, while map explorer will get you working with 2-dimensional arrays.

>



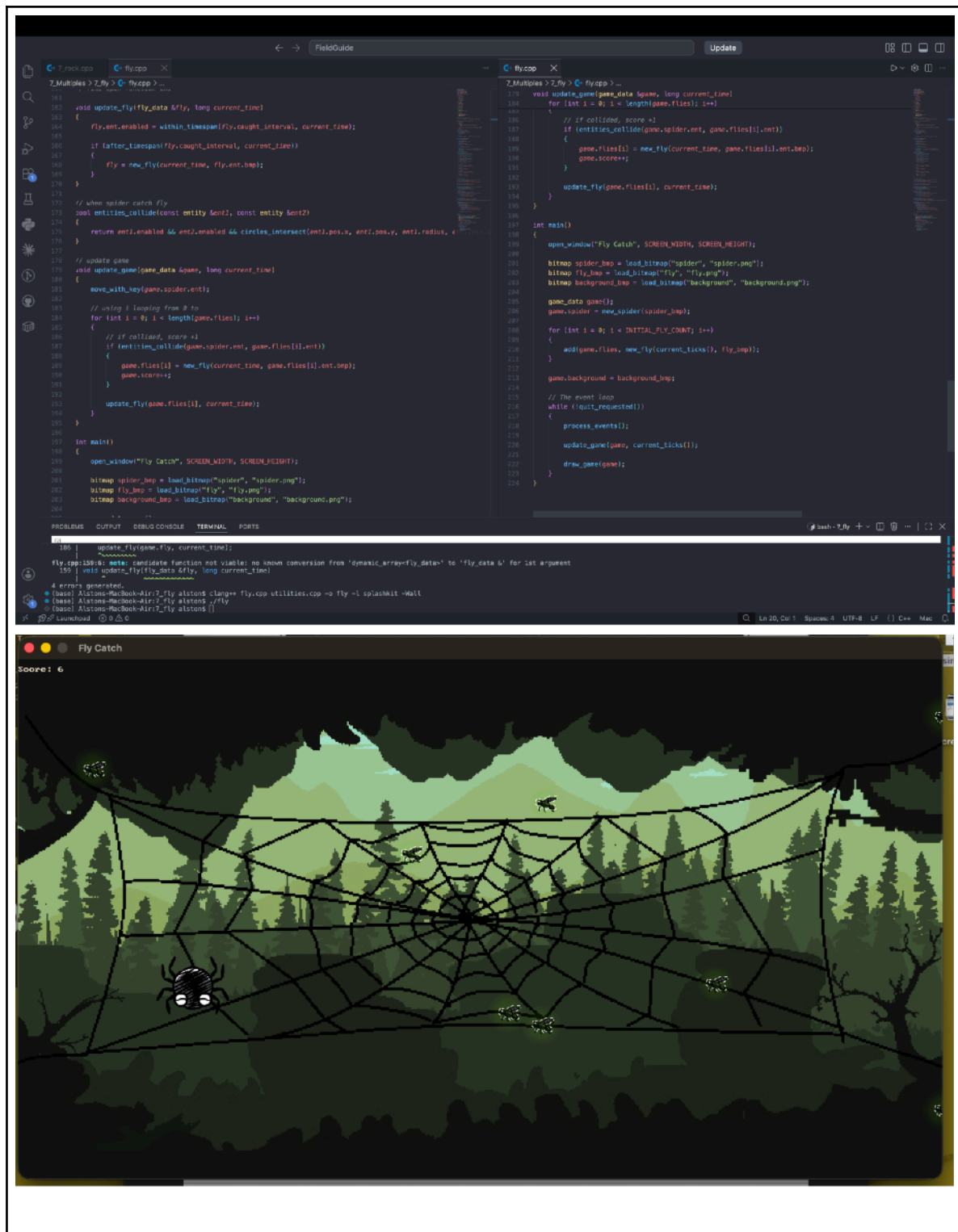
```

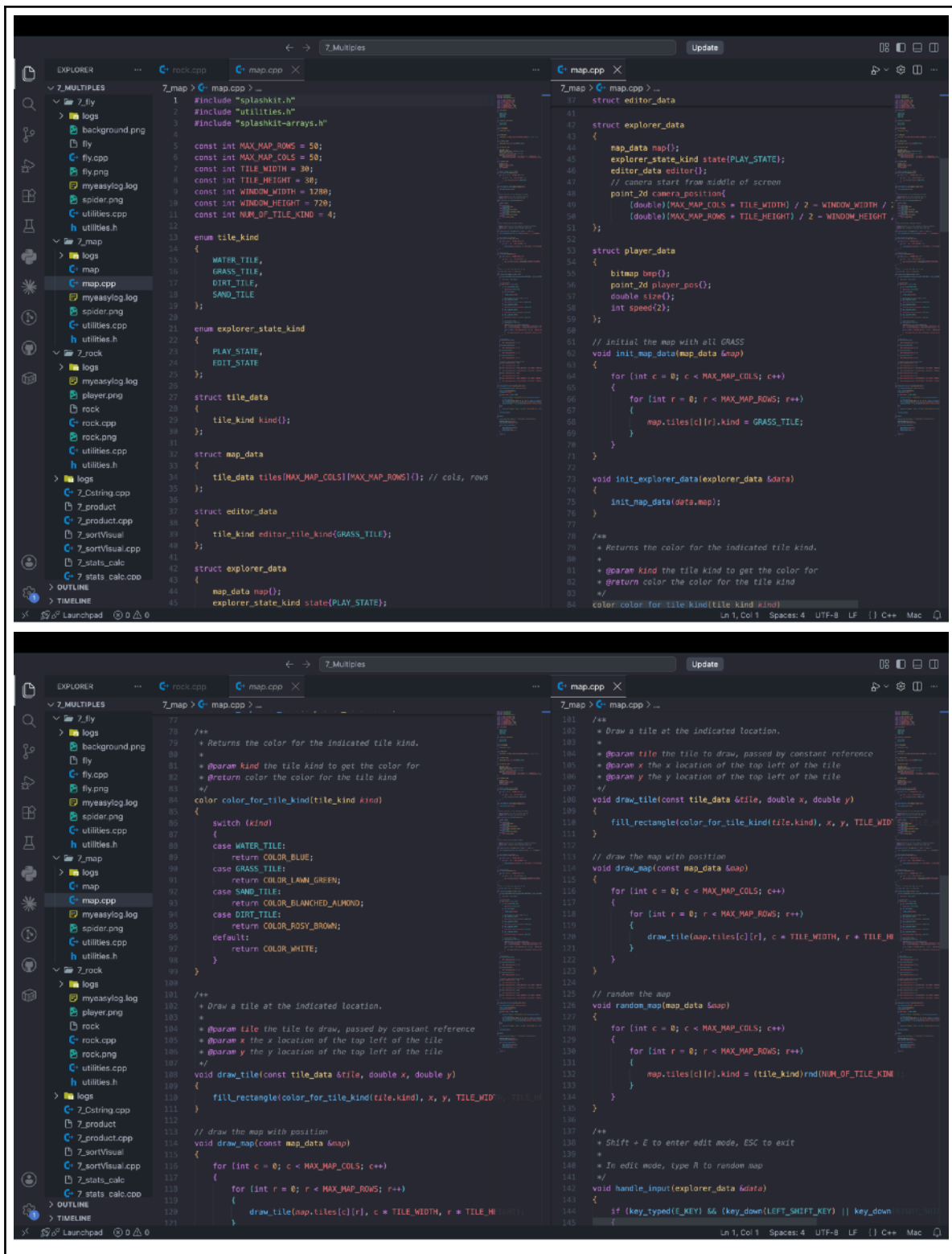
7_Multiples.cpp
1 #include "splashkit.h"
2 #include "utilities.h"
3 #include "splashkit-arrays.h"
4
5 const int SCREEN_WIDTH = 1280;
6 const int SCREEN_HEIGHT = 720;
7 const int SPEED = 3;
8 const int INITIAL_FLY_COUNT = 10;
9
10 struct entity
11 {
12     bool enabled();
13     point_2d pos();
14     double radius();
15
16     bitmap bmp();
17     int speed(SPEED);
18 };
19
20 struct timespan
21 {
22     long start();
23     long duration();
24 };
25
26 struct spider_data
27 {
28     entity ent();
29 };
30
31 struct fly_data
32 {
33     entity ent();
34     timespan caught_interval();
35 };
36
37 struct game_data
38 {
39     spider_data spider();
40     dynamic_array<fly_data> flies();
41     bitmap background();
42     int score();
43 };
44
45
46 fly.cpp
1 struct fly_data
2 {
3     entity ent();
4     timespan caught_interval();
5 };
6
7 struct spider_data
8 {
9     entity ent();
10 };
11
12 struct game_data
13 {
14     spider_data spider();
15     dynamic_array<fly_data> flies();
16     bitmap background();
17     int score();
18 };
19
20 void draw_game(const game_data &game)
21 {
22     clear_screen(COLOR_WHITE);
23
24     // Draw background
25     draw_bitmap(game.background, 0, 0);
26
27     // Draw spider
28     draw_entity(game.spider.ent);
29
30     for (int i = 0; i < length(game.flies); i++)
31     {
32         // Draw fly
33         draw_entity(game.flies[i].ent);
34     }
35
36     // draw the score
37     draw_text("Score: " + ta_string(game.score), COLOR_ANTIQUE_WHITE, 0, 10);
38     refresh_screen();
39 }
40
41 void move_with_key(entity &ent)
42 {
43     if (key_down(RIGHT_KEY) || key_down(D_KEY))
44     {
45         ent.pos.x += ent.speed;
46     }
47     if (key_down(LEFT_KEY) || key_down(A_KEY))
48     {
49         ent.pos.x -= ent.speed;
50     }
51     if (key_down(DOWN_KEY) || key_down(S_KEY))
52     {
53         ent.pos.y += ent.speed;
54     }
55     if (key_down(UP_KEY) || key_down(W_KEY))
56     {
57         ent.pos.y -= ent.speed;
58     }
59
60     // check right
61     if (ent.pos.x + ent.radius > SCREEN_WIDTH)
62     {
63         ent.pos.x = SCREEN_WIDTH - 2 * ent.radius;
64     }
65     // check left
66     if (ent.pos.x - ent.radius < 0)
67     {
68         ent.pos.x = 2 * ent.radius;
69     }
70     // check down
71     if (ent.pos.y + ent.radius > SCREEN_HEIGHT)
72     {
73         ent.pos.y = SCREEN_HEIGHT - 2 * ent.radius;
74     }
75     // check up
76     if (ent.pos.y - ent.radius < 0)
77     {
78         ent.pos.y = 2 * ent.radius;
79     }
80 }
81
82 // Time span function start
83
84 // full within timespan span.start == time < (span.start + span.duration)
85 bool within_timespan(const timespan &span, long time)
86 {
87     return (time == span.start) && (time < (span.start + span.duration));
88 }
89
90 bool after_timespan(const timespan &span, long time)
91 {
92     return time >= (span.start + span.duration);
93 }
94
95 // overloading : use a easier version of a function to call a full version
96 bool within_timespan(const timespan &span)
97 {
98     return within_timespan(span, current_ticks());
99 }
100
101 // Time span function end

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

185 | update\_fly(game.fly, current\_time);  
fly.cpp:159:8: note: candidate function not viable: no known conversion from 'dynamic\_array<fly\_data>' to 'fly\_data &' for 1st argument  
159 | void update\_fly(fly\_data &fly, long current\_time)  
4 errors generated.  
[base] Alton-MacBook-Air:7\_fly alton\$ clang++ fly.cpp utilities.cpp -o fly -l splashkit -Wall  
[base] Alton-MacBook-Air:7\_fly alton\$ ./fly  
[base] Alton-MacBook-Air:7\_fly alton\$ |  
Launchpad

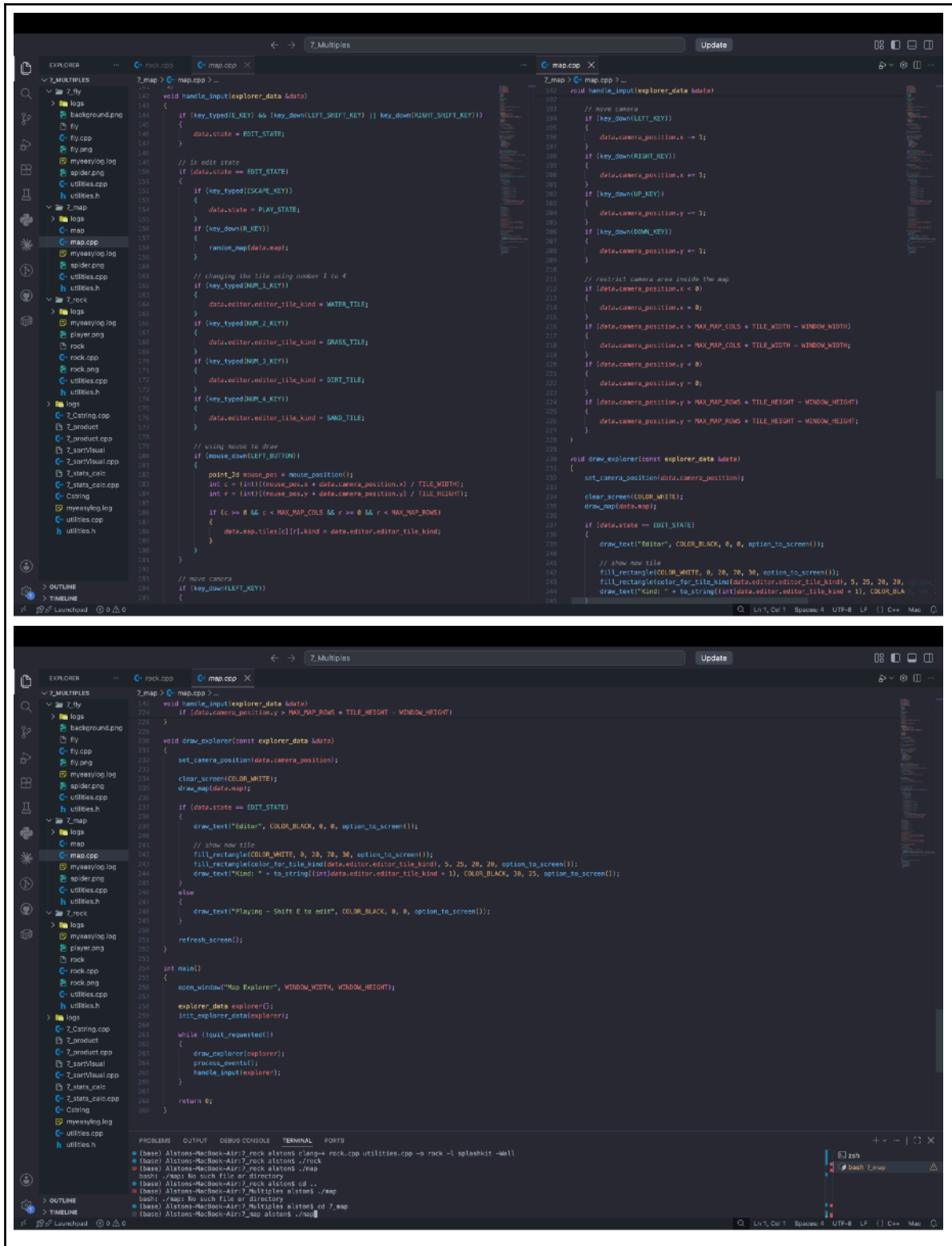




```

1  #include "splashkit.h"
2  #include "utilities.h"
3  #include "splashkit-arrays.h"
4
5  const int MAX_MAP_ROWS = 50;
6  const int MAX_MAP_COLS = 50;
7  const int TILE_WIDTH = 30;
8  const int TILE_HEIGHT = 30;
9  const int WINDOW_WIDTH = 1280;
10 const int WINDOW_HEIGHT = 720;
11 const int NUM_OF_TILE_KIND = 4;
12
13 enum tile_kind
14 {
15     WATER_TILE,
16     GRASS_TILE,
17     DIRT_TILE,
18     SAND_TILE
19 };
20
21 enum explorer_state_kind
22 {
23     PLAY_STATE,
24     EDIT_STATE
25 };
26
27 struct tile_data
28 {
29     tile_kind kind();
30 };
31
32 struct map_data
33 {
34     tile_data tiles[MAX_MAP_COLS][MAX_MAP_ROWS](); // cols, rows
35 };
36
37 struct editor_data
38 {
39     tile_kind editor_tile_kind(GRASS_TILE);
40 };
41
42 struct explorer_data
43 {
44     map_data map();
45     explorer_state_kind state(PLAY_STATE);
46
47     struct editor_data
48     {
49         map_data map();
50         explorer_state_kind state(PLAY_STATE);
51         editor_data editor();
52         // camera start from middle of screen
53         point_2d camera_position{
54             (double)(MAX_MAP_COLS * TILE_WIDTH) / 2 - WINDOW_WIDTH / 2,
55             (double)(MAX_MAP_ROWS * TILE_HEIGHT) / 2 - WINDOW_HEIGHT / 2
56         };
57     };
58
59     struct player_data
60     {
61         bitmap bmp();
62         point_2d player_pos();
63         double size();
64         int speed(2);
65     };
66
67     // initial the map with all GRASS
68     void init_map_data(map_data &map)
69     {
70         for (int c = 0; c < MAX_MAP_COLS; c++)
71         {
72             for (int r = 0; r < MAX_MAP_ROWS; r++)
73             {
74                 map.tiles[c][r].kind = GRASS_TILE;
75             }
76         }
77     }
78
79     void init_explorer_data(explorer_data &data)
80     {
81         init_map_data(data.map);
82     }
83
84     // Returns the color for the indicated tile kind.
85     // @param kind the tile kind to get the color for
86     // @return color the color for the tile kind
87     color color_for_tile_kind(tile_kind kind)
88     {
89         switch (kind)
90         {
91             case WATER_TILE:
92                 return COLOR_BLUE;
93             case GRASS_TILE:
94                 return COLOR_LAWN_GREEN;
95             case SAND_TILE:
96                 return COLOR_BLANCHED_ALMOND;
97             case DIRT_TILE:
98                 return COLOR_ROSY_BROWN;
99             default:
100                 return COLOR_WHITE;
101         }
102     }
103
104     // Draw a tile at the indicated location.
105     // @param tile the tile to draw, passed by constant reference
106     // @param x the x location of the top left of the tile
107     // @param y the y location of the top left of the tile
108     void draw_tile(const tile_data &tile, double x, double y)
109     {
110         fill_rectangle(color_for_tile_kind(tile.kind), x, y, TILE_WIDTH, TILE_HEIGHT);
111     }
112
113     // draw the map with position
114     void draw_map(const map_data &map)
115     {
116         for (int c = 0; c < MAX_MAP_COLS; c++)
117         {
118             for (int r = 0; r < MAX_MAP_ROWS; r++)
119             {
120                 draw_tile(map.tiles[c][r], c * TILE_WIDTH, r * TILE_HEIGHT);
121             }
122         }
123     }
124
125     // random the map
126     void random_map(map_data &map)
127     {
128         for (int c = 0; c < MAX_MAP_COLS; c++)
129         {
130             for (int r = 0; r < MAX_MAP_ROWS; r++)
131             {
132                 map.tiles[c][r].kind = (tile_kind)rand(NUM_OF_TILE_KIND);
133             }
134         }
135     }
136
137     // Shift + E to enter edit mode, ESC to exit
138     // In edit mode, type A to random map
139     void handle_input(explorer_data &data)
140     {
141         if (key_typed(E_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
142         {
143             //
144         }
145     }

```



The image shows two screenshots of a Visual Studio Code editor with a C++ project named 'Z\_Multiples'. The left screenshot shows the implementation of the `handle_input` function in `map.cpp`, which handles various keyboard inputs for editing the map state. The right screenshot shows the `draw_explorer` function, which renders the map and UI elements.

```

// map.cpp
142 void handle_input(explorer_data &data)
143 {
144     if (key_pressed(L_KEY) && (key_down(LEFT_SHIFT_KEY) || key_down(RIGHT_SHIFT_KEY)))
145     {
146         data.state = EDIT_STATE;
147     }
148     // in edit state
149     if (data.state == EDIT_STATE)
150     {
151         if (key_pressed(ESCAPE_KEY))
152         {
153             data.state = PLAY_STATE;
154         }
155         if (key_down(R_KEY))
156         {
157             random_map(data.map);
158         }
159         // changing the tile using number 1 to 4
160         if (key_pressed(MAP_1_KEY))
161         {
162             data.editor.editor_tile_kind = WATER_TILE;
163         }
164         if (key_pressed(MAP_2_KEY))
165         {
166             data.editor.editor_tile_kind = GRASS_TILE;
167         }
168         if (key_pressed(MAP_3_KEY))
169         {
170             data.editor.editor_tile_kind = DIRT_TILE;
171         }
172         if (key_pressed(MAP_4_KEY))
173         {
174             data.editor.editor_tile_kind = SAND_TILE;
175         }
176         // using mouse to draw
177         if (mouse_down(LEFT_BUTTON))
178         {
179             point_2d mouse_pos = mouse_position();
180             int c = (int)((mouse_pos.x + data.camera_position.x) / TILE_WIDTH);
181             int r = (int)((mouse_pos.y + data.camera_position.y) / TILE_HEIGHT);
182             if (c >= 0 && c < MAX_MAP_COLS && r >= 0 && r < MAX_MAP_ROWS)
183             {
184                 data.map.tiles[c][r].kind = data.editor.editor_tile_kind;
185             }
186         }
187     }
188     // move camera
189     if (key_down(LEFT_KEY))
190     {
191         // move camera
192         if (key_down(LEFT_KEY))
193         {
194             // move camera
195             if (key_down(LEFT_KEY))
196             {
197                 data.camera_position.x -= 1;
198             }
199             if (key_down(RIGHT_KEY))
200             {
199                 data.camera_position.x += 1;
200             }
201             if (key_down(UP_KEY))
202             {
203                 data.camera_position.y -= 1;
204             }
205             if (key_down(DOWN_KEY))
206             {
207                 data.camera_position.y += 1;
208             }
209         }
210         // restrict camera area inside the map
211         if (data.camera_position.x < 0)
212         {
213             data.camera_position.x = 0;
214         }
215         if (data.camera_position.x > MAX_MAP_COLS + TILE_WIDTH - WINDOW_WIDTH)
216         {
217             data.camera_position.x = MAX_MAP_COLS + TILE_WIDTH - WINDOW_WIDTH;
218         }
219         if (data.camera_position.y < 0)
220         {
221             data.camera_position.y = 0;
222         }
223         if (data.camera_position.y > MAX_MAP_ROWS + TILE_HEIGHT - WINDOW_HEIGHT)
224         {
225             data.camera_position.y = MAX_MAP_ROWS + TILE_HEIGHT - WINDOW_HEIGHT;
226         }
227     }
228 }
229 void draw_explorer(const explorer_data &data)
230 {
231     set_camera_position(data.camera_position);
232     clear_screen(COLOR_WHITE);
233     draw_map(data.map);
234     if (data.state == EDIT_STATE)
235     {
236         draw_text("Editor", COLOR_BLACK, 0, 0, option_to_screen());
237         // show new tile
238         fill_rectangle(COLOR_WHITE, 0, 20, 30, 30, option_to_screen());
239         fill_rectangle(COLOR_WHITE, 5, 25, 20, 20, option_to_screen());
240         draw_text("kind = " + to_string((int)data.editor.editor_tile_kind + 1), COLOR_BLACK, 30, 25, option_to_screen());
241     }
242     else
243     {
244         draw_text("Playing - Shift E to edit", COLOR_BLACK, 0, 0, option_to_screen());
245     }
246     refresh_screen();
247 }
248 int main()
249 {
250     open_window("Map Explorer", WINDOW_WIDTH, WINDOW_HEIGHT);
251     explorer_data explorer;
252     init_explorer_data(explorer);
253     while (!quit_requested())
254     {
255         draw_explorer(explorer);
256         process_events();
257         handle_input(explorer);
258     }
259     return 0;
260 }

```

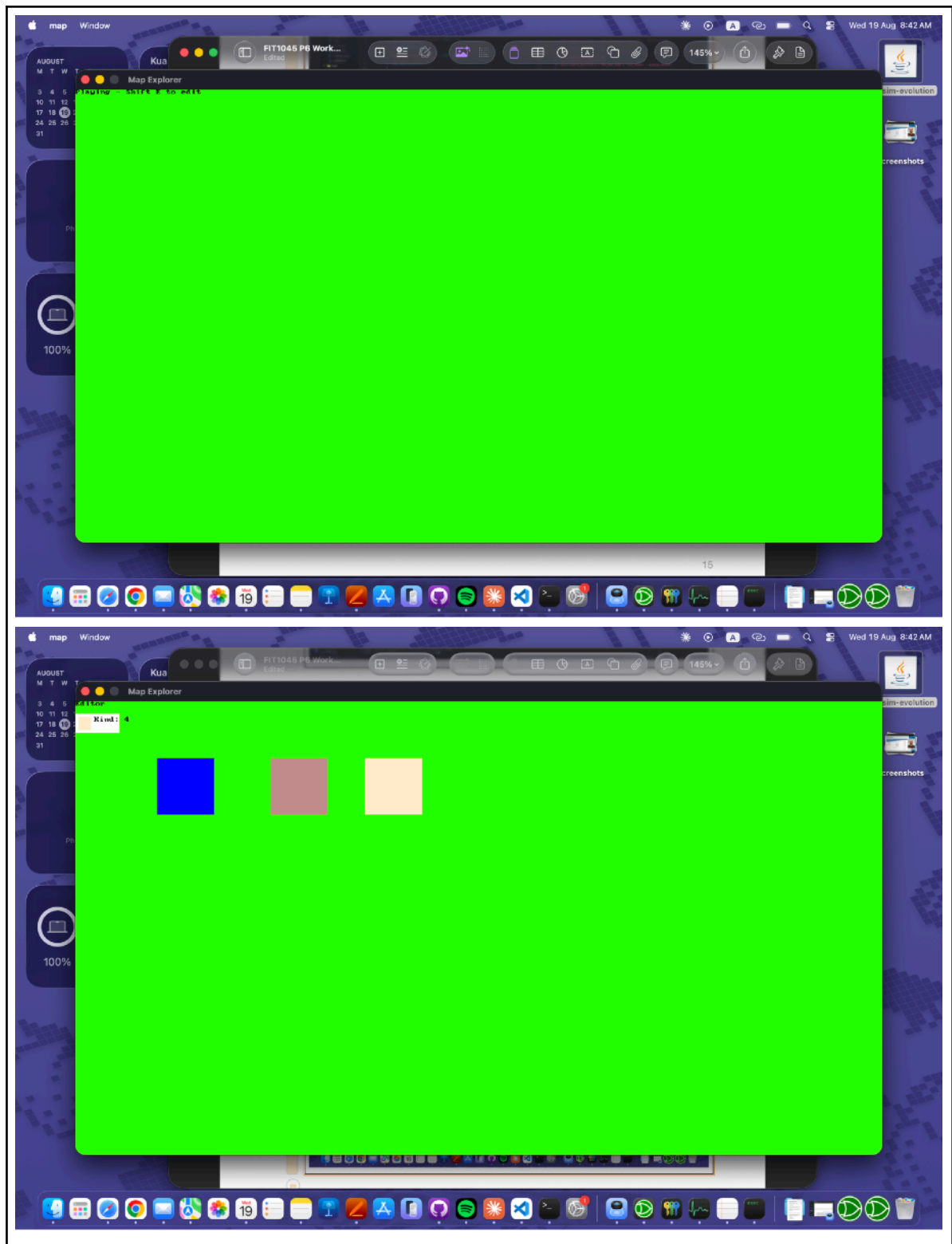
The right screenshot shows the `draw_explorer` function, which renders the map and UI elements. It includes logic for displaying the current editor state, showing the selected tile kind, and refreshing the screen.

```

// map.cpp
230 void draw_explorer(const explorer_data &data)
231 {
232     set_camera_position(data.camera_position);
233     clear_screen(COLOR_WHITE);
234     draw_map(data.map);
235     if (data.state == EDIT_STATE)
236     {
237         draw_text("Editor", COLOR_BLACK, 0, 0, option_to_screen());
238         // show new tile
239         fill_rectangle(COLOR_WHITE, 0, 20, 30, 30, option_to_screen());
240         fill_rectangle(COLOR_WHITE, 5, 25, 20, 20, option_to_screen());
241         draw_text("kind = " + to_string((int)data.editor.editor_tile_kind + 1), COLOR_BLACK, 30, 25, option_to_screen());
242     }
243     else
244     {
245         draw_text("Playing - Shift E to edit", COLOR_BLACK, 0, 0, option_to_screen());
246     }
247     refresh_screen();
248 }
249 int main()
250 {
251     open_window("Map Explorer", WINDOW_WIDTH, WINDOW_HEIGHT);
252     explorer_data explorer;
253     init_explorer_data(explorer);
254     while (!quit_requested())
255     {
256         draw_explorer(explorer);
257         process_events();
258         handle_input(explorer);
259     }
260     return 0;
261 }

```

The bottom screenshot shows the terminal output, indicating that the program was compiled successfully and is running. The output shows the command `rock -o splashkit -Wall` and the resulting executable `rock`.

**Reflect:**

- What was the most interesting thing you learnt through this task, and why?

> I learned about how to build a complex and huge project step by step. First need to build a frame about this project, separate a big problem to many small problem. Then for each small problem, we need to think how to solve it, can use function, procedure or otherwise to solve it. The combine all the solution to become a full version program do complete the initial goal.

- How are arrays and loops connected?

> Arrays in c++ is very inconvenient to do some operation. Using loop we can do some operation like reading, adding, removing or use it. Every time using loop in array, need to using int  $i$  to looping arrays[ $i$ ].

- Looking back, how long did it take to complete this task? What did you spend the most time on?

> I use 32 hours for this task. I many time to doing map explorer and rock dodge. When I finish the initial version of rock dodge using circle, I want to changing it to sprite, I face many problem so then I use bitmap to replace sprite but it also cannot work, the position of rock and player get wrong and I cannot fix it. I debug very long time to fix this.

## Disclose:

- Explain your use of Generative AI for this task. If you didn't use it, just write "I did not use any AI for this task". If you did use it, please state which AI model you used (e.g. ChatGPT) and how.

> I use Claude to guide me doing the map explorer to add some function.  
In the Rock dodge game, I ask about how to remove the marked rock from the array and also help me to check the position problem.

## Submit:

- Export this worksheet as a PDF.
- Upload your submission to OnTrack:
- Engage with feedback and discussion to get this signed off.